ChatGPT

M2.8 Camera, Lighting and Responsive Layout

Executive summary

Assumption: target = mid-range iOS/Android; goal = 60fps on modern phones and acceptable 30fps fallback on low-end. Best fit is **keep Flutter UI architecture, add faux-3D camera via `Transform/Matrix4`, board split into render layers, and use `FragmentProgram` only for lighting/material passes**. This matches the current architecture, reuses the asset pipeline, avoids native-engine overhead, and respects Flutter's strengths: adaptive layout, shader assets, image caching, and profile tooling. Flutter fragment shaders are supported, but **Flutter does not support vertex shaders**, so true mesh/perspective deformation should not be the primary plan. [1]

Goals, constraints and deliverables

Goals: camera tilt/perspective, adaptive phone/tablet layout, layered lighting/materials, asset-driven board depth, and measurable frame budgets. Constraints: **no engine/gameplay changes**, Flutter-only presentation, reuse existing asset pipeline, and avoid heavy native embeds. Deliverables: camera prototype, responsive spec, lighting options matrix, material atlas, shader/asset-pipeline additions, tests, and a performance budget. Flutter's responsive guidance favors `MediaQuery.sizeOf`, `LayoutBuilder`, and `SafeArea`; performance validation should be on real devices in profile mode. [2]

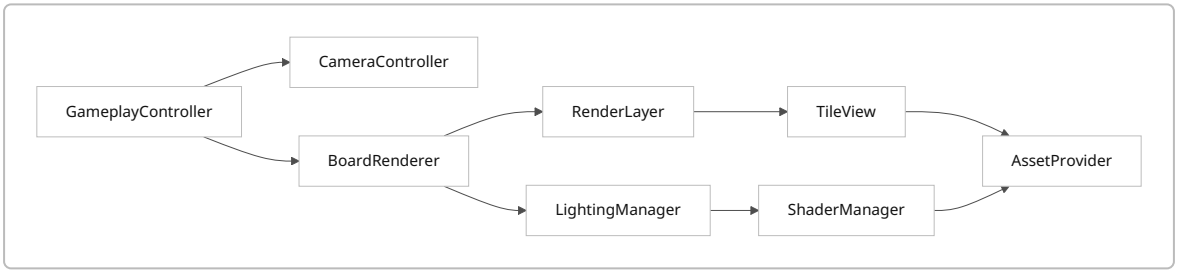
Options and recommendation

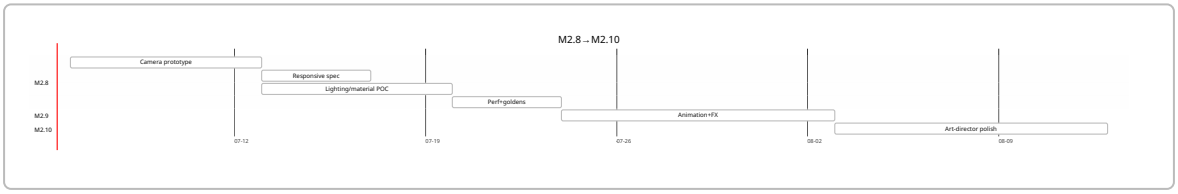
Option	Pros	Cons	Fit
Widgets+`Transform`+`CustomPainter`	Lowest risk; keeps current tree	Limited true 3D	**Best**
FragmentProgram/SkSL	Strong lighting/material polish	No vertex shaders; watch raster cost	**Best adjunct**
Flame	Built-in camera/layers/snapshots	Introduces game-loop/world model drift	Medium
Rive	Excellent micro-anim/shared textures	Not ideal for board camera/material system	Low-medium
Unity embed	True 3D	Platform-view perf and architectural cost	**Avoid**

Flame offers `CameraComponent`, `World`, and render layers/snapshots; Rive recommends `RiveWidget`/shared textures; platform views have documented performance tradeoffs on the platform thread. [3]

Shader path	Use	Risk
Per-tile fragment lighting	normals/AO/rim	Moderate raster cost
Full-board post pass	bloom/deferred-lite	Good if bounded
CPU-painted gradients only	safest	weaker premium look

Technical design






```
final m = Matrix4.identity()
  ..setEntry(3, 2, 0.0012) // perspective
  ..rotateX(0.92)          // tilt
  ..scale(boardScale);
```

```
vec3 n=normalize(texNormal*2.0-1.0);
float ndl=max(dot(n,uLightDir),0.0);
float rim=pow(1.0-max(dot(n,uViewDir),0.0),2.0);
vec3 lit=albedo*(uAmbient+ndl*uDiffuse)+rim*uRim;
```

Assets, performance and responsive rules

Required textures: albedo/diffuse, normals, AO, roughness/specular, height maps; atlas tiles by biome/object. Prefer predecoded/cached images, bounded decode sizes (`ResizeImage`), and isolate static board/HUD with `RepaintBoundary`; Flutter's global `ImageCache` defaults to 1000 images/100MB and should be tuned. Budget: modern ≤16ms/frame; low-end degrade by disabling bloom, lowering shader samples, and using flatter materials. [4]

Responsive rules: width-based breakpoints, not device type/orientation; phone = single-column HUD, tablet = side HUD; compute tile size from local constraints via `LayoutBuilder`; use `SafeArea` for

cutouts. Accessibility: preserve contrast, non-colour cues, motion reduction, and avoid camera tilt that harms target highlighting. [5]

Testing, migration and roadmap

Tests: unit (`CameraController`, breakpoint math), widget (HUD repositioning), goldens (phone/tablet/light presets), performance (profile-mode frame chart/UI vs raster), device matrix (low/mid/high Android + iPhone). Migrate incrementally: **M2.8** camera/responsive/material foundation → **M2.9** animation and richer FX → **M2.10** art-direction pass. Effort: camera medium, responsive low, material atlas medium, shader stack high, perf/goldens medium. Recommended tools: Flutter DevTools Performance/CPU profiler, `FragmentProgram`, `CustomPainter`, `LayoutBuilder`, `SafeArea`, `RepaintBoundary`; optionally Flame snapshots only if caching pressure grows. [6]

---

[1] Writing and using fragment shaders
https://docs.flutter.dev/ui/design/graphics/fragment-shaders

[2] General approach to adaptive apps
https://docs.flutter.dev/ui/adaptive-responsive/general

[3] Camera & World — Flame
https://docs.flame-engine.org/latest/flame/camera.html

[4] ImageCache class - painting library - Dart API
https://api.flutter.dev/flutter/painting/ImageCache-class.html

[5] Adaptive and responsive design in Flutter
https://docs.flutter.dev/ui/adaptive-responsive

[6] Use the Performance view
https://docs.flutter.dev/tools/devtools/performance